Equivalence Class Partitioning and Boundary Value Analysis

October 13th, 2025
Ana Paiva, Auri Vincenzi, José Campos

In this recitation class, we are going to explore ‘Equivalence Class Partitioning’ and ‘Boundary Value Analysis’, *two black-box testing techniques*, in the `jpacman` project (available here).

⚠️ Please make sure your machine is configured properly, i.e., a bash-based shell, Java JDK 11, Apache Maven v3.9.11, and that you have the latest version of IntelliJ IDEA installed. If not, please visit the 0. Setup document.

1. Perform ‘Equivalence Class Partitioning’ and ‘Boundary Value Analysis’

Given the brief and short “specification” of the `jpacman` project, which you could find here, we expect you to perform ‘Equivalence Class Partitioning’ and ‘Boundary Value Analysis’ on the following 5 functions.

In a nutshell, apply Equivalence Class Partitioning and Boundary Value Analysis, as described in the lecture class, to the inputs of the functions described below. Then, write down in a `txt` file (1) the partitions you managed to find according to the Equivalence Class Partitioning’s procedure and (2) the inputs you are going to use to test each partition according to the Boundary Value Analysis’ procedure.

1.1 `squareAt` in the `nl.tudelft.jpacman.board.Board` class

```java
/**
 * Returns the square at the given <code>x,y</code> position.
 *
 * Precondition: The <code>(x, y)</code> coordinates are within the
 * width and height of the board.
 *
 * @param x
```

```
 *   The <code>x</code> position (column) of the requested square.
 * @param y
 *   The <code>y</code> position (row) of the requested square.
 * @return The square at the given <code>x,y</code> position (never
 * null).
 */
public Square squareAt(int x, int y)
```

1.2 `createBoard` in the `nl.tudelft.jpacman.board.BoardFactory` class

Java

```
/**
 * Creates a new board from a grid of cells and connects it.
 *
 * @param grid
 *   The square grid of cells, in which grid[x][y] corresponds to
the
 *   square at position x,y.
 * @return A new board, wrapping a grid of connected cells.
 */
public Board createBoard(Square[][] grid)
```

1.3 `createLevel` in the `nl.tudelft.jpacman.level.LevelFactory` class

Java

```
/**
 * Creates a new level from the provided data.
 *
 * @param board
 *   The board with all ghosts and pellets occupying their squares.
 * @param ghosts
 *   A list of all ghosts on the board.
```

```
 * @param startPositions
 *   A list of squares from which players may start the game.
 * @return A new level for the board.
 */
public Level createLevel(Board board, List<Ghost> ghosts,
List<Square> startPositions)
```

1.4 `makeGhostSquare` in the `nl.tudelft.jpacman.level.MapParser` class

Java

```
/**
 * Creates a Square with the specified ghost on it and appends the
 * placed ghost into the ghost list.
 *
 * @param ghosts
 *   All the ghosts in the level so far, the new ghost will be
appended.
 * @param ghost
 *   The newly created ghost to be placed.
 * @return a square with the ghost on it.
 */
protected Square makeGhostSquare(List<Ghost> ghosts, Ghost ghost)
```

1.5 `draw` in the `nl.tudelft.jpacman.sprite.ImageSprite` class

Java

```
/**
 * Draws the sprite on the provided graphics context.
 *
 * @param graphics
 *   The graphics context to draw.
 * @param x
```

```
 *   The destination x coordinate to start drawing.
 * @param y
 *   The destination y coordinate to start drawing.
 * @param width
 *   The width of the destination draw area.
 * @param height
 *   The height of the destination draw area.
 */
public void draw(Graphics graphics, int x, int y, int width, int
height)
```

2. Write unit tests

Write unit test cases using the JUnit framework to every single combination/input you defined in Section 1. Perform 'Equivalence Class Partitioning' and 'Boundary Value Analysis'.

⚠️Note: in Maven-based projects, tests must be developed under `src/test/java`. For more information on how to write JUnit test cases or using Mockito visit the 0. JUnit document or 0. Mockito document, respectively.

Miscellaneous

- JUnit framework
- JUnit 5 User Guide